

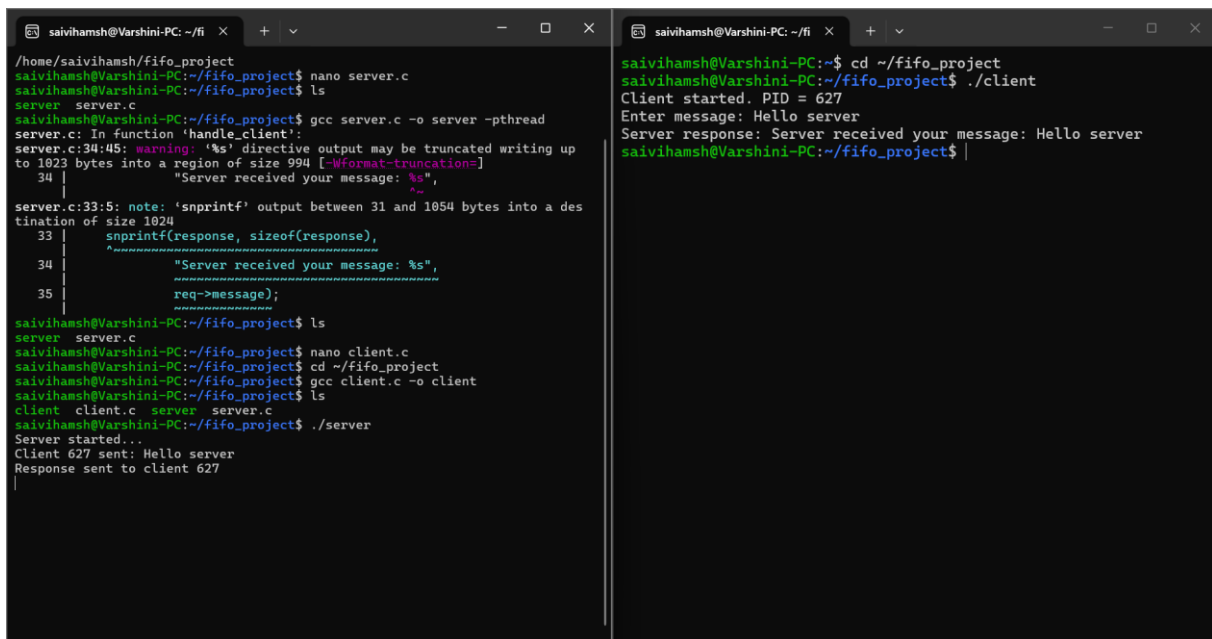
PRACTICAL WEEK – 6

NAME:CH.SAI VIHAMSH

ROLL:2520090090

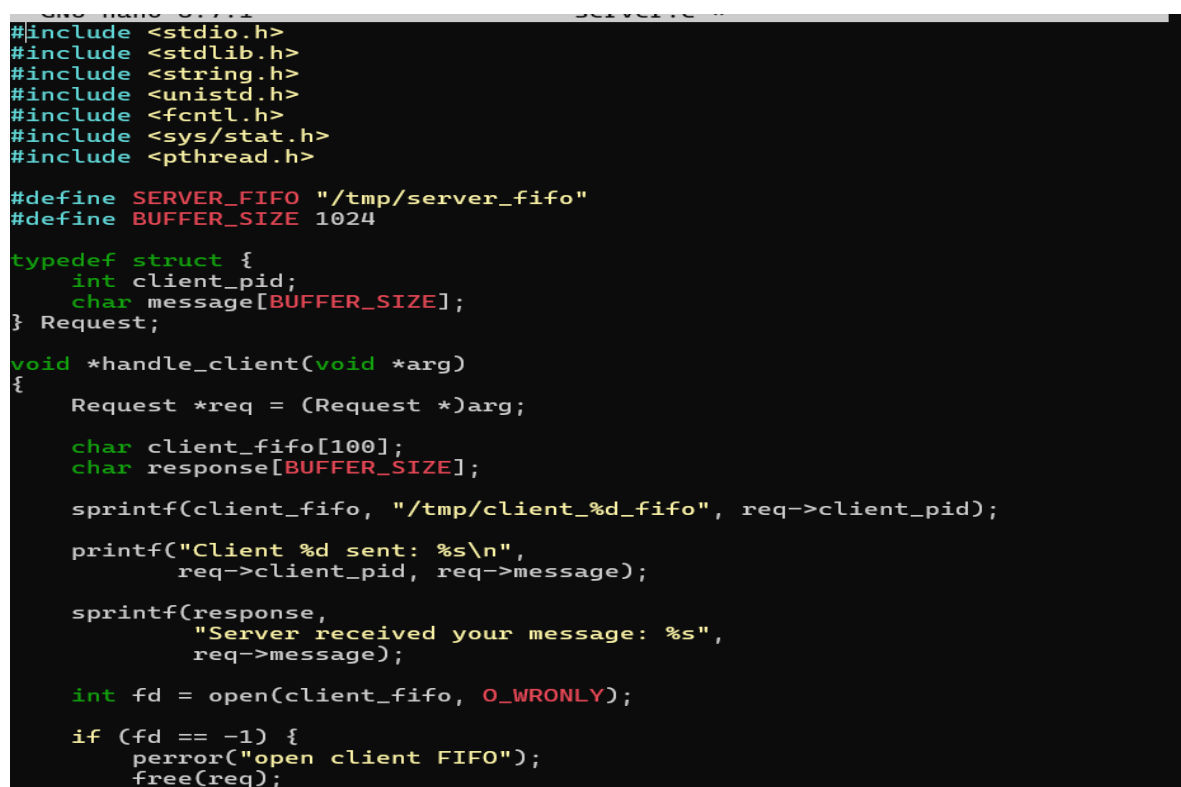
SESSION-6

Task-1: Create a client-server application using Named Pipes (FIFOs). The client sends messages to the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.



```
saivihamsh@Varshini-PC: ~/fi x + v - □ X
/home/saivihamsh/fifo_project
saivihamsh@Varshini-PC:~/fifo_project$ nano server.c
saivihamsh@Varshini-PC:~/fifo_project$ ls
server server.c
saivihamsh@Varshini-PC:~/fifo_project$ gcc server.c -o server -pthread
server.c: In function 'handle_client':
server.c:34:45: warning: 's' directive output may be truncated writing up
to 1823 bytes into a region of size 994 [-Wformat-truncation=]
34 |         "Server received your message: %s",
    |         ^~
server.c:33:5: note: 'sprintf' output between 31 and 1854 bytes into a des
tination of size 1024
33 |         sprintf(response, sizeof(response),
    |         ^~~~~~
34 |         "Server received your message: %s",
    |         ^~
35 |         req->message);
    |         ^~~~~~
saivihamsh@Varshini-PC:~/fifo_project$ ls
server server.c
saivihamsh@Varshini-PC:~/fifo_project$ nano client.c
saivihamsh@Varshini-PC:~/fifo_project$ cd ~/fifo_project
saivihamsh@Varshini-PC:~/fifo_project$ gcc client.c -o client
saivihamsh@Varshini-PC:~/fifo_project$ ls
client client.c server server.c
saivihamsh@Varshini-PC:~/fifo_project$ ./server
Server started...
Client 627 sent: Hello server
Response sent to client 627

saivihamsh@Varshini-PC:~/fi x + v - □ X
saivihamsh@Varshini-PC:~$ cd ~/fifo_project
saivihamsh@Varshini-PC:~/fifo_project$ ./client
Client started. PID = 627
Enter message: Hello server
Server response: Server received your message: Hello server
saivihamsh@Varshini-PC:~/fifo_project$ |
```



```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <pthread.h>

#define SERVER_FIFO "/tmp/server_fifo"
#define BUFFER_SIZE 1024

typedef struct {
    int client_pid;
    char message[BUFFER_SIZE];
} Request;

void *handle_client(void *arg)
{
    Request *req = (Request *)arg;

    char client_fifo[100];
    char response[BUFFER_SIZE];

    sprintf(client_fifo, "/tmp/client_%d_fifo", req->client_pid);
    printf("Client %d sent: %s\n",
        req->client_pid, req->message);

    sprintf(response,
        "Server received your message: %s",
        req->message);

    int fd = open(client_fifo, O_WRONLY);

    if (fd == -1) {
        perror("open client FIFO");
        free(req);
    }
}
```

```

        printf("Response sent to Client %d\n", req->client_pid);

        free(req);

        return NULL;
    }
}

int main()
{
    printf("Server started...\n");

    // Create server FIFO
    mkfifo(SERVER_FIFO, 0666);

    // Open FIFO for reading and writing
    int fd = open(SERVER_FIFO, O_RDWR);

    if (fd == -1) {
        perror("open server FIFO");
        exit(1);
    }

    while (1)
    {
        Request *req = malloc(sizeof(Request));

        if (req == NULL) {
            perror("malloc");
            continue;
        }

        ssize_t bytes = read(fd, req, sizeof(Request));

        if (bytes <= 0) {
            free(req);
            continue;
        }
    }
}

```

```

GNU nano 8.7.1 client.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

#define SERVER_FIFO "/tmp/server_fifo"
#define BUFFER_SIZE 1024

typedef struct {
    int client_pid;
    char message[BUFFER_SIZE];
} Request;

int main()
{
    int pid = getpid();

    char client_fifo[100];

    sprintf(client_fifo, "/tmp/client_%d_fifo", pid);

    // Create client FIFO
    mkfifo(client_fifo, 0666);

    printf("Client started. PID = %d\n", pid);

    char message[BUFFER_SIZE];

    printf("Enter message: ");
    fgets(message, BUFFER_SIZE, stdin);

    message[strcspn(message, "\n")] = '\0';

    Request req;

```

```
GNU nano 2.7.1 client.c *
// Open server FIFO
int server_fd = open(SERVER_FIFO, O_WRONLY);

if (server_fd == -1) {
    perror("open server FIFO");
    unlink(client_fifo);
    exit(1);
}

// Send request
write(server_fd, &req, sizeof(Request));

close(server_fd);

// Open client FIFO to receive response
int client_fd = open(client_fifo, O_RDONLY);

if (client_fd == -1) {
    perror("open client FIFO");
    unlink(client_fifo);
    exit(1);
}

char response[BUFFER_SIZE];

read(client_fd, response, BUFFER_SIZE);

printf("Server response: %s\n", response);

close(client_fd);

// Delete client FIFO
unlink(client_fifo);

return 0;
}
```

Task2: Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
saivihamsh@Varshini-PC: ~/si + ▾  
saivihamsh@Varshini-PC:~/signal_project$ ./signals  
Signal handling program started.  
Process ID (PID): 551  
  
Send signals using:  
SIGINT : Ctrl+C  
SIGTERM : kill -SIGTERM <PID>  
SIGUSR1 : kill -SIGUSR1 <PID>  
  
Program is running...  
Program is running...  
Program is running...  
  
SIGUSR1 received  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
  
SIGTERM received  
Program is running...  
Program is running...  
Program is running...  
^C  
SIGINT received (Ctrl+C)  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
^Z  
[1]+ Stopped ./signals  
saivihamsh@Varshini-PC:~/signal_project$ kill -9 $!  
[1]+ Stopped ./signals  
saivihamsh@Varshini-PC:~/signal_project$ ^C  
[1]+ Killed  
saivihamsh@Varshini-PC:~/signal_project$ ls  
signal_handler.c signals signal.c  
saivihamsh@Varshini-PC:~/signal_project$
```

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>

volatile sig_atomic_t sigint_count = 0;
volatile sig_atomic_t sigterm_count = 0;
volatile sig_atomic_t sigusr1_count = 0;

void handle_signal(int signal)
{
    if (signal == SIGINT)
    {
        sigint_count++;

        write(STDOUT_FILENO,
              "\nSIGINT received (Ctrl+C)\n",
              26);
    }

    else if (signal == SIGTERM)
    {
        sigterm_count++;

        write(STDOUT_FILENO,
              "\nSIGTERM received\n",
              19);
    }

    else if (signal == SIGUSR1)
    {
        sigusr1_count++;

        write(STDOUT_FILENO,
              "\nSIGUSR1 received\n",
              19);
    }
}

```

```

sa.sa_handler = handle_signal;

sigemptyset(&sa.sa_mask);

sa.sa_flags = 0;

if (sigaction(SIGINT, &sa, NULL) == -1)
{
    perror("sigaction SIGINT");
    exit(1);
}

if (sigaction(SIGTERM, &sa, NULL) == -1)
{
    perror("sigaction SIGTERM");
    exit(1);
}

if (sigaction(SIGUSR1, &sa, NULL) == -1)
{
    perror("sigaction SIGUSR1");
    exit(1);
}

printf("Signal handling program started.\n");
printf("Process ID (PID): %d\n", getpid());

printf("\nSend signals using:\n");
printf("SIGINT   : Ctrl+C\n");
printf("SIGTERM  : kill -SIGTERM %d\n", getpid());
printf("SIGUSR1   : kill -SIGUSR1 %d\n\n", getpid());

while (1)
{
    printf("Program is running...\n");
    sleep(5);
}

```

Observation: The POSIX signal handling program successfully captured SIGINT, SIGTERM, and SIGUSR1 signals. The signal handler responded asynchronously whenever a signal was received, while the program continued its execution. Thus, asynchronous event handling using POSIX signals was successfully demonstrated.